LAB MANUAL
PART A
(PART A: TO BE REFERRED BY STUDENTS)
Experiment No: -10

Aim:- Implementation and analysis of the Rabin-Karp string matching algorithm.

Resource Required:- Any C compiler, VSCode or online compiler.
Theory:-

Rabin-Karp Algorithm:
1. Rabin-Karp is a string-matching algorithm that searches for a pattern in a text by comparing hash values instead of comparing every character at every position.
1. It computes a hash value for the pattern and for each window of the text having the same length as the pattern.

String Matching Problem:
1. Given a text T of length n and a pattern P of length m, the objective is to find all positions in T where P occurs.
1. If the hash value of a text window matches the pattern hash, the characters are compared to confirm the match because different strings may produce the same hash value.
1. A rolling hash is used to efficiently calculate the hash of the next text window from the previous window.

Hash Function:
Pattern hash: p = (d * p + pattern[i]) mod q
Text-window hash: t = (d * t + text[i]) mod q
Rolling hash: t(next) = (d * (t - text[i] * h) + text[i + m]) mod q
where d is the number of possible characters, q is a prime number, and h = d^(m-1) mod q.

Algorithm:-
RabinKarp(text, pattern, q)
let n = length of text and m = length of pattern
compute h = d^(m - 1) mod q
compute the initial hash values of the pattern and first text window
for i = 0 to n - m
if pattern hash equals text-window hash
compare pattern characters with the current text window
if all characters match, report index i
if i < n - m, calculate the hash of the next text window using rolling hash
if the new text hash is negative, add q to make it positive

C Implementation:-
#include <stdio.h>

```
#include <string.h>

#define D 256

void rabinKarp(char text[], char pattern[], int q) {
    int n = strlen(text);
    int m = strlen(pattern);
    int p = 0;
    int t = 0;
    int h = 1;

    for (int i = 0; i < m - 1; i++)
        h = (h * D) % q;

    for (int i = 0; i < m; i++) {
        p = (D * p + pattern[i]) % q;
        t = (D * t + text[i]) % q;
    }

    for (int i = 0; i <= n - m; i++) {
        if (p == t) {
            int j;
            for (j = 0; j < m; j++) {
                if (text[i + j] != pattern[j])
                    break;
            }
            if (j == m)
                printf("Pattern found at index %d\n", i);
        }

        if (i < n - m) {
            t = (D * (t - text[i] * h) + text[i + m]) % q;
            if (t < 0)
                t = t + q;
        }
    }
}

int main() {
    char text[] = "AABAACAADAABAABA";
    char pattern[] = "AABA";
    int q = 101;

    printf("Text    : %s\n", text);
```

```
    printf("Pattern : %s\n", pattern);
    rabinKarp(text, pattern, q);

    return 0;
}
```

OUTPUT:

```
Text    : AABAACAADAABAABA
Pattern : AABA
Pattern found at index 0
Pattern found at index 9
Pattern found at index 12
```

Dry Run Example — Text = "AABAACAADAABAABA", Pattern = "AABA":

1. The pattern length is 4, so Rabin-Karp computes the hash of "AABA" and the first 4-character text window.
2. At index 0, the hash values match and character comparison confirms that "AABA" occurs.
3. The rolling hash is then used to calculate the hash for each next window without recomputing it from the beginning.
4. Further confirmed matches are found at indices 9 and 12.

Step	Window Index	Text Window	Hash Comparison	Result
1	0	AABA	Equal	Pattern found
2	1	ABAA	Not equal	No match
3	2	BAAC	Not equal	No match
4	9	AABA	Equal	Pattern found
5	12	AABA	Equal	Pattern found

Complexity Analysis:

1. Best and average-case time complexity is O(n + m) when the hash function distributes values well and only a small number of hash collisions occur.
2. Worst-case time complexity is O(nm) when many text windows have the same hash value as the pattern and character-by-character verification is repeatedly required.
3. Auxiliary space complexity is O(1), excluding the input text and pattern.

Algorithm	Best Case	Average Case	Worst Case
Rabin-Karp	O(n + m)	O(n + m)	O(nm)

Conclusion:

You should have learned how the Rabin-Karp algorithm uses hashing and a rolling hash to search for a pattern in a text, verifies possible matches, reports matching indices, and analyzes its time and space requirements.

PART B

(PART B: TO BE COMPLETED BY STUDENTS)

(Students must submit the soft copy as per the following segments within two hours of the practical. The soft copy must be uploaded on the Blackboard or emailed to the concerned lab in-charge faculty at the end of the practical in case there is no Blackboard access available.)

Roll No.	C-60	**Name:**	Avdhut Madhavi
Class	COMP-C	**Batch:**	C2
Date of Experiment:		**Date of Submission:**	
Grade:			

B.1: Implement and trace the following programs. Write the code, compile/run it, and record the output after each task.

1. Run the Rabin-Karp program using a different text and pattern. Record all matching indices and verify the output manually.

```
#include <stdio.h>
#include <string.h>

#define d 256
#define q 101

void rabinKarp(char text[], char pattern[])
{
   int n = strlen(text);
   int m = strlen(pattern);
   int i, j, p = 0, t = 0, h = 1;
   int found = 0;

   if (m == 0 || m > n)
   {
      printf("Pattern not found.\n");
      return;
   }

   for (i = 0; i < m - 1; i++)
      h = (h * d) % q;

   for (i = 0; i < m; i++)
   {
      p = (d * p + pattern[i]) % q;
      t = (d * t + text[i]) % q;
   }

   for (i = 0; i <= n - m; i++)
```

```
    {
        if (p == t)
        {
            for (j = 0; j < m; j++)
            {
                if (text[i + j] != pattern[j])
                    break;
            }

            if (j == m)
            {
                printf("Pattern found at index %d\n", i);
                found = 1;
            }
        }

        if (i < n - m)
        {
            t = (d * (t - (text[i] * h) % q)
                 + text[i + m]) % q;

            if (t < 0)
                t += q;
        }
    }

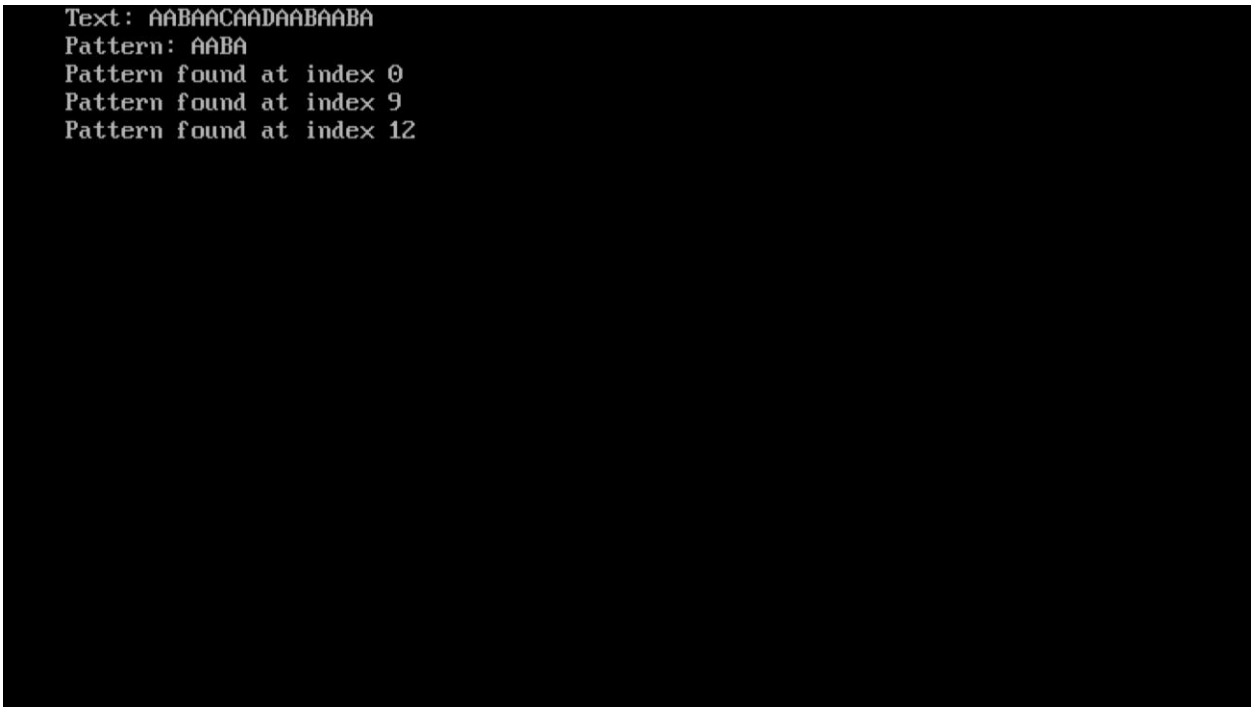
    if (!found)
        printf("Pattern not found.\n");
}

int main()
{
    char text[] = "AABAACAADAABAABA";
    char pattern[] = "AABA";

    printf("Text: %s\n", text);
    printf("Pattern: %s\n", pattern);

    rabinKarp(text, pattern);

    return 0;
}
```



2. Modify the program to accept the text and pattern from the user and display all occurrences of the pattern.

```
#include <stdio.h>
#include <string.h>

#define d 256
#define q 101

void rabinKarp(char text[], char pattern[])
{
   int n, m, i, j;
   long p = 0, t = 0, h = 1;
   int found = 0;

   n = strlen(text);
   m = strlen(pattern);

   if (m == 0 || m > n)
   {
      printf("Pattern not found.\n");
      return;
   }

   for (i = 0; i < m - 1; i++)
```

```
        h = (h * d) % q;

    for (i = 0; i < m; i++)
    {
        p = (d * p + pattern[i]) % q;
        t = (d * t + text[i]) % q;
    }

    for (i = 0; i <= n - m; i++)
    {
        if (p == t)
        {
            for (j = 0; j < m; j++)
            {
                if (text[i + j] != pattern[j])
                    break;
            }

            if (j == m)
            {
                printf("Pattern found at index %d\n", i);
                found = 1;
            }
        }

        if (i < n - m)
        {
            t = (d * (t - (text[i] * h) % q)
                 + text[i + m]) % q;

            if (t < 0)
                t += q;
        }
    }

    if (found == 0)
        printf("Pattern not found.\n");
}

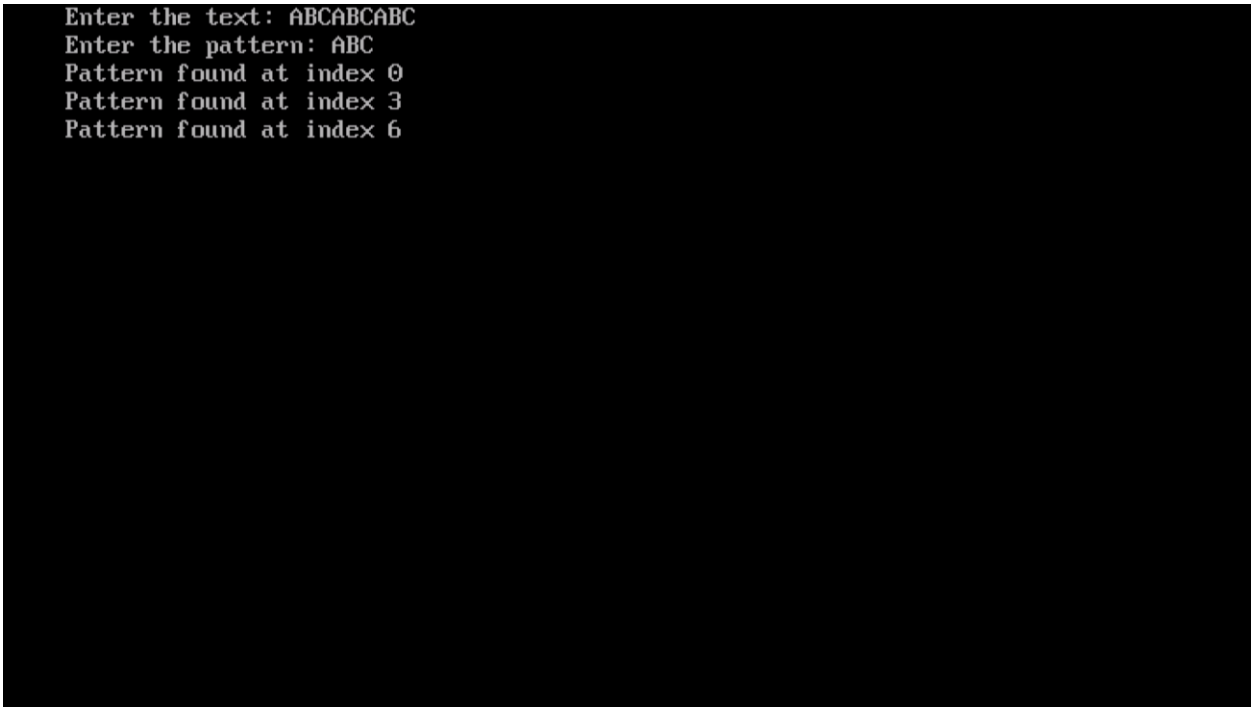
int main()
{
    char text[1000], pattern[1000];

    printf("Enter the text: ");
```

```
    gets(text);

    printf("Enter the pattern: ");
    gets(pattern);

    rabinKarp(text, pattern);
    getch();
    return 0;
}
```



3. Do the analysis of the Rabin-Karp string matching algorithm and compare its best, average, and worst-case time complexities.

1. Introduction

The Rabin-Karp algorithm is a string-matching algorithm used to find the occurrence of a pattern within a given text. It uses a hashing technique to compare the hash value of the pattern with the hash value of each substring of the text.

Let:

- (n) = length of the text
- (m) = length of the pattern

2. Working of the algorithm

1. Calculate the hash value of the pattern.
2. Calculate the hash value of the first substring of length \(m\) in the text.
3. Compare both hash values.
4. If the hash values match, compare the actual characters to confirm the match.

5. Use a rolling hash to calculate the hash of the next substring efficiently.
6. Repeat until all possible positions in the text are checked.

3. Time complexity analysis

Case	Time complexity	Explanation
Best case	(O(n+m))	Hash calculations and rolling hash updates take linear time, with no costly character-by-character verification.
Average case	(O(n+m))	Hashing usually eliminates most unnecessary character comparisons, assuming few collisions and a good hash function.
Worst case	(O(nm))	Hash collisions or repeated matching prefixes may cause up to (m) character comparisons at each of approximately (n) positions.

Note: Since (m<n) for a possible match, the best and average cases are also commonly written as (O(n)).

4. Space complexity

The algorithm uses a constant amount of extra space for hash values and loop variables, excluding the input strings.
O(1) auxiliary space

5. Advantages

1. Efficient for searching patterns in large texts.
2. Rolling hash avoids recalculating the entire substring hash at each position.
3. Useful for multiple-pattern matching and plagiarism detection.
4. Easy to implement.

6. Conclusion

The Rabin-Karp algorithm uses hashing to perform string matching efficiently. Its best and average time complexity is (O(n+m)), while its worst-case time complexity is \(O(nm)\) because of hash collisions and repeated character comparisons.